Introduction to Agentic AI and CodeTrain

Understanding AI Agents and Workflow Orchestration

Speedykom Group GmbH

2026-04-16

1. What Are AI Agents?

The Evolution: From Scripts to Agents

Think about how you've written code so far. Traditional programs follow a fixed path:

1. **Input** → User provides data
2. **Process** → Code executes predetermined steps
3. **Output** → Result is returned

The program never asks questions mid-execution, never looks up additional information, and never adapts its approach based on what it discovers.

AI Agents change this paradigm entirely.

An AI Agent is an autonomous system that can:

- **Perceive** its environment (read files, query APIs, browse the web)
- **Reason** about what it observes (use LLMs to make decisions)
- **Act** on its reasoning (execute code, call tools, make changes)
- **Learn** from outcomes (adapt its strategy based on results)

The Key Difference

Traditional Code	AI Agent
Fixed logic paths	Dynamic decision-making
Needs exact instructions	Figures out "how" from "what"
Fails on edge cases	Adapts and retries
No external awareness	Can browse, query, and research
Stateless	Maintains context and memory

Real-World Example

Imagine building a system to summarize news articles:

Traditional Approach:

```
# Hard-coded steps
def summarize_article(url):
    html = fetch_url(url)           # Step 1
    text = extract_text(html)       # Step 2
    summary = run_summarizer(text)  # Step 3
    return summary
```

Agentic Approach:

```
# Dynamic decision-making
class ArticleAgent(Agent):
    def run(self, topic):
        # Agent decides what to do
        search_results = self.search_web(topic)
        relevant_urls = self.filter_relevant(search_results)

        summaries = []
        for url in relevant_urls:
            content = self.fetch_and_extract(url)
            # Agent decides how deeply to analyze
            if self.is_complex_topic(content):
                summary = self.deep_analysis(content)
            else:
                summary = self.quick_summary(content)
            summaries.append(summary)

        # Agent synthesizes final output
        return self.synthesize_report(summaries)
```

The agent doesn't just follow steps—it **decides** what steps are needed.

Anatomy of an Agent

Every AI Agent has four core components:

1. The Brain (LLM)

The Large Language Model provides reasoning capabilities. It can:

- Understand natural language instructions
- Generate code and text

- Make decisions based on context
- Break complex tasks into subtasks

2. Tools

Agents use tools to interact with the world:

- **File operations** (read, write, edit)
- **Web access** (search, browse, API calls)
- **Code execution** (run Python, shell commands)
- **Memory** (store and retrieve information)

3. Planning System

The agent decides what to do next:

- **Sequential**: Do A, then B, then C
- **Conditional**: If X, do Y; otherwise do Z
- **Iterative**: Keep trying until success
- **Hierarchical**: Break big tasks into smaller ones

4. Memory

Agents remember context across interactions:

- **Short-term**: What just happened in this conversation
- **Long-term**: Facts learned from previous sessions
- **Working memory**: Current task context and intermediate results

The AI Agent Ecosystem: Building the "Mech Suit" for the LLM

To understand AI tools, you first have to understand the fundamental relationship between a Large Language Model (LLM) and an Agent Framework.

- **The LLM (The Brain in a Jar):** Models like GPT-4, Claude, or Gemini are brilliant but isolated. They understand complex concepts and can write code, but they have no memory, no "hands" to type, and no "eyes" to read your screen. They only process text in and text out.
- **The Agent Framework (The Mech Suit):** This is the software built *around* the LLM. It provides the "senses" (reading your files or messages), the "hands" (running terminal commands or sending emails), and the "hippocampus" (storing conversation memory).

The framework manages a continuous loop: It gathers context → prompts the LLM → parses the LLM's requested action → executes the tool on your machine → feeds the result back to the LLM → repeats.

2. What Is Orchestration?

Beyond Single Agents

A single agent is powerful, but real-world problems often require multiple specialized agents working together. **Orchestration** is the coordination of multiple agents (or tasks) into a cohesive workflow.

Think of it like a delivery operation:

- One person receives orders
- Another prepares packages
- Another handles shipping
- Another manages customer service

Each person is an "agent" with a specific job. The **orchestrator** ensures they work together smoothly.

Why Orchestration Matters

1. Separation of Concerns

Breaking complex workflows into discrete steps makes them:

- **Easier to debug**: Know exactly which step failed
- **Easier to maintain**: Update one step without breaking others
- **Easier to scale**: Add more capacity to bottleneck steps
- **Easier to understand**: Each step has a clear purpose

2. Reliability

When tasks are modular, you can:

- Retry failed steps independently
- Add error handling at specific points
- Monitor each step's performance
- Replace failing components without system-wide changes

3. Flexibility

Orchestrated workflows can:

- Branch based on conditions (if paid, ship express; else ship standard)
- Loop until success (retry failed API calls)
- Run in parallel (process multiple items simultaneously)
- Adapt to different scenarios (different paths for different data types)

4. Reusability

Once you build a "Send Email" task, you can reuse it in:

- Order confirmations
- Password resets
- Newsletter delivery
- Alert notifications

Orchestration Patterns

Sequential Workflow

Tasks execute one after another:

```
Receive Order → Validate Payment → Prepare Package → Ship Order
```

Conditional Workflow

Different paths based on conditions:

```
                    → Standard Shipping
Receive Order → Check Type
                    → Express Shipping
```

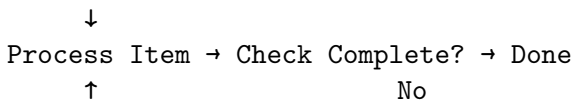
Map-Reduce Workflow

Process items in parallel, then combine results:

```
Input List → [Process A] [Process B] [Process C] → Combine Results
               ↓           ↓           ↓
             Item 1      Item 2      Item 3
```

Loop Workflow

Repeat until a condition is met:



3. The Landscape: Finished Cars vs. Engine Blocks

The "Finished Cars" (Pre-Built Agents)

The current heavyweight open-source tools generally fall into two distinct categories:

1. Personal AI Assistants (The 24/7 Digital Chiefs of Staff)

These live in messaging apps (Telegram, Slack) or run as background cron jobs to automate your life.

- **OpenClaw (The Juggernaut):** A massive (~430k lines) Node.js agent with a huge plugin ecosystem. It can do almost anything but is a massive security risk if not strictly sandboxed (e.g., in Docker), as it wants full system access.
- **Nanobot (The Minimalist):** A lightweight (~4k lines) Python alternative. It replicates OpenClaw's core features (messaging integrations, web search) but uses minimal RAM. It's perfect for deploying on cheap cloud servers or a Raspberry Pi, though it lacks a fancy GUI or massive plugin store.

2. Dedicated Coding Agents (The Software Engineers)

These live in your terminal or IDE and are strictly focused on building and refactoring software.

- **OpenCode (The Powerhouse):** A Go-based agent with a modern Terminal UI (TUI), Desktop app, and deep IDE integration. It uses Language Server Protocol (LSP) to understand complex project architectures and supports 75+ local and cloud LLMs.
- **Aider (The Terminal Gold Standard):** A Python-based, terminal-only tool famous for its safe, Git-native workflow. Every change it makes is a clean, reviewable Git commit. It is the industry benchmark for pair-programming safely but requires you to be comfortable in the CLI.

The "Engine Block" (CodeTrain)

If you don't want to buy a "finished car," you can build your own custom agent using **CodeTrain**.

Inspired by the 100-line PocketFlow framework, CodeTrain is a minimalist orchestration engine that uses a logistics-based architecture to handle complex AI logic without the bloat of traditional libraries.

Why Build with CodeTrain?

- **Total Ownership:** You understand every line of your agent's routing logic.
- **Agentic Self-Looping:** Easily build chatbots where a Job loops back to itself (`chat_job - "continue" >> chat_job`) while the Manifest maintains persistent conversation history.
- **SKILL.md Integration:** You can feed the CodeTrain `SKILL.md` to a "Finished Car" like **OpenCode** and tell it: *"Use this architecture to build me a specialized `TranslationJob`."* The AI will build the code perfectly every time.

The CodeTrain Architecture (Logistics Pattern)

1. **The Manifest (The Cargo):** A shared dictionary that flows through the entire system, carrying all context, data, and memory.
2. **The Jobs (The Workstations):** Specialized units of work with a strict three-stage lifecycle:
   - `receive_order()`: Load and validate input from the manifest.
   - `prepare_order()`: Execute the actual work (e.g., call the LLM).
   - `ship_order()`: Finalize and store the results back in the manifest.
3. **The Hustle (The Route):** The overarching recipe that connects Jobs using the `>>` operator for sequences and `- "action" >>` for conditional branching.

Summary Checklist for Your Build

If you want to build...	CodeTrain handles...	You will need to build...
A Terminal Agent (like Aider)	Three-stage job logic, LLM routing, manifest state	Terminal UI, Git auto-commits, file Read/Write
An IDE Agent (like Cursor)	Workflow orchestration, conditional branching	VS Code extension, Diffing UI (Red/Green code)
A Personal Bot (like Nanobot)	Self-looping loops, context memory, API calls	Messaging API hooks (Telegram), cron jobs

Core Philosophy

> "The best framework is the one that gets out of your way and lets you focus on solving business problems, not wrestling with complex abstractions."

CodeTrain follows the PocketFlow principle: **minimal code, maximum expressiveness**.

4. Installing and Setting Up CodeTrain

Installation

We use `uv` for package management in this course.

Using uv (Recommended)

```
uv pip install git+https://github.com/Speedykom/CodeTrain.git
```

Alternative: Using pip

If you don't have `uv` installed:

```
pip install git+https://github.com/Speedykom/CodeTrain.git
```

Verify Installation

```
import codetrain as ct
print(ct.__version__)
```

> **Virtual Environment Recommended**
>
> Always install in a virtual environment to avoid conflicts:
>
> ```
> # Using uv (recommended)
> uv venv
> source .venv/bin/activate  # On Windows: .venv\Scripts\activate
> uv pip install git+https://github.com/Speedykom/CodeTrain.git
>
> # Alternative: Using standard Python
> python -m venv venv
> source venv/bin/activate  # On Windows: venv\Scripts\activate
> pip install git+https://github.com/Speedykom/CodeTrain.git
> ```

5. CodeTrain Fundamentals

Understanding Jobs

A **Job** is the basic unit of work in CodeTrain. Every Job has three stages:

1. `receive_order(manifest)` → Load and validate input data
2. `prepare_order(data)` → Execute the actual work
3. `ship_order(manifest, data, result)` → Finalize and store output

This mirrors a delivery logistics workflow:

- **Receive**: Accept the package order
- **Prepare**: Package and label the item
- **Ship**: Hand off to delivery and update tracking

Your First Job

```
import codetrain as ct

class GreetingJob(ct.Job):
    """A simple job that greets someone."""
```

```
    def receive_order(self, manifest):
        """Load data from the manifest."""
        # Get the name from manifest, default to "World"
        name = manifest.get("name", "World")
        return name

    def prepare_order(self, name):
        """Do the actual work."""
        # Create a greeting message
        greeting = f"Hello, {name}!"
        return greeting

    def ship_order(self, manifest, name, greeting):
        """Store the result back in the manifest."""
        manifest["greeting"] = greeting
        manifest["processed_at"] = "just now"
        return "done"

# Create and run the job
job = GreetingJob()
manifest = {"name": "Ghana"}
job.run(manifest)

print(manifest["greeting"])  # Output: Hello, Ghana!
print(manifest["processed_at"])  # Output: just now
```

> **i** The Manifest Pattern
>
> The manifest is a shared dictionary that flows through your entire workflow. Jobs can: - **Read** from it (in `receive_order`) - **Write** to it (in `ship_order`) - **Pass data** between stages via return values

Connecting Jobs into Workflows

Single jobs are useful, but the real power comes from **chaining jobs together**. CodeTrain uses the `>>` operator to connect jobs:

```
import codetrain as ct

class ExtractNameJob(ct.Job):
    """Extracts a name from user input."""

    def receive_order(self, manifest):
        return manifest.get("user_input", "")
```

```
    def prepare_order(self, user_input):
        # Simple extraction: assume input is the name
        return user_input.strip()

    def ship_order(self, manifest, user_input, name):
        manifest["name"] = name
        return "done"

class ValidateNameJob(ct.Job):
    """Validates that a name is not empty."""

    def receive_order(self, manifest):
        return manifest.get("name", "")

    def prepare_order(self, name):
        if len(name) < 2:
            raise ValueError("Name must be at least 2 characters")
        return name.upper()

    def ship_order(self, manifest, name, validated_name):
        manifest["validated_name"] = validated_name
        manifest["is_valid"] = True
        return "done"

class CreateWelcomeMessageJob(ct.Job):
    """Creates a personalized welcome message."""

    def receive_order(self, manifest):
        return manifest.get("validated_name", "Guest")

    def prepare_order(self, name):
        return f"Welcome, {name}! We're excited to have you."

    def ship_order(self, manifest, name, message):
        manifest["welcome_message"] = message
        return "done"

# Connect jobs into a workflow
extract = ExtractNameJob()
validate = ValidateNameJob()
welcome = CreateWelcomeMessageJob()

# Create the workflow using >>
workflow = extract >> validate >> welcome

# Run with a hustle
```

```
hustle = ct.Hustle(start=workflow)
manifest = {"user_input": "  kwame  "}
hustle.run(manifest)

print(manifest["name"])              # Output: kwame
print(manifest["validated_name"])    # Output: KWAME
print(manifest["welcome_message"])   # Output: Welcome, KWAME! We're excited to have you.
```

> **The » Operator**
>
> Think of `>>` as "then" or "passes to":
>
> - `job_a >> job_b` means "do job_a, then pass the manifest to job_b"
> - You can chain as many jobs as needed: `a >> b >> c >> d`
> - Each job receives the same manifest, but updated by previous jobs

Conditional Routing

Sometimes you need different paths based on conditions. CodeTrain supports this with the `-"action" >>` syntax:

```
import codetrain as ct

class CheckAgeJob(ct.Job):
    """Checks if user is an adult."""

    def receive_order(self, manifest):
        return manifest.get("age", 0)

    def prepare_order(self, age):
        return age >= 18

    def ship_order(self, manifest, age, is_adult):
        manifest["is_adult"] = is_adult
        manifest["age"] = age
        # Return action for conditional routing
        return "adult" if is_adult else "minor"

class AdultContentJob(ct.Job):
    """Shows adult-appropriate content."""

    def receive_order(self, manifest):
        return manifest.get("name", "User")

    def prepare_order(self, name):
```

```
        return f"Welcome to the adult section, {name}!"

    def ship_order(self, manifest, name, message):
        manifest["content"] = message
        return "done"

class MinorContentJob(ct.Job):
    """Shows child-appropriate content."""

    def receive_order(self, manifest):
        return manifest.get("name", "User")

    def prepare_order(self, name):
        return f"Welcome to the kids section, {name}!  "

    def ship_order(self, manifest, name, message):
        manifest["content"] = message
        return "done"

# Create jobs
check = CheckAgeJob()
adult_content = AdultContentJob()
minor_content = MinorContentJob()

# Set up conditional workflow
# If check returns "adult", go to adult_content
# If check returns "minor", go to minor_content
check - "adult" >> adult_content
check - "minor" >> minor_content

# Create hustle starting at check
hustle = ct.Hustle(start=check)

# Test with adult
manifest1 = {"name": "Kwame", "age": 25}
hustle.run(manifest1)
print(f"Adult: {manifest1['content']}")

# Test with minor
manifest2 = {"name": "Ama", "age": 12}
hustle.run(manifest2)
print(f"Minor: {manifest2['content']}")
```

! Action-Based Routing

The `- "action" >>` syntax connects a specific return value to a specific job:

- Job returns `"action_name"` → Goes to connected job
- This enables branching workflows (if/else logic)
- Multiple actions can be defined for different branches

6. Building a Chatbot with CodeTrain

One of the most powerful features of CodeTrain is the ability to create **self-looping workflows**. This is perfect for building interactive chatbots that maintain conversation history and continue running until the user decides to exit.

The Self-Looping Pattern

In CodeTrain, a Job can connect to itself using the `- "action" >>` syntax:

```
chat_job - "continue" >> chat_job  # Job loops back to itself
```

This creates an infinite conversation loop that runs until the Job returns `None` to signal completion. The key insight is that the **manifest persists across iterations**, allowing you to maintain state (like conversation history) throughout the chat session.

Simple Keyword-Based Chatbot

Let's build a simple chatbot that responds based on keywords:

```
import codetrain as ct

class SimpleChatJob(ct.Job):
    """A simple chatbot with keyword responses."""

    def receive_order(self, manifest):
        """Initialize conversation history and get user input."""
        # Initialize messages list on first run
        if "messages" not in manifest:
            manifest["messages"] = []
            print("=" * 60)
            print("  Welcome to CodeTrain Chatbot!")
            print("=" * 60)
            print("Type 'exit' to end the conversation")
            print("-" * 60)

        # Get user input
```

```
        user_input = input("\nYou: ").strip()

        # Check if user wants to exit
        if user_input.lower() == "exit":
            return None

        # Add user message to history
        manifest["messages"].append({"role": "user", "content": user_input})

        return user_input

    def prepare_order(self, user_input):
        """Generate a response based on keywords."""
        if user_input is None:
            return None

        user_text = user_input.lower()

        # Keyword-based responses
        responses = {
            "hello": "Hello there! How can I help you today?  ",
            "hi": "Hi! What can I do for you?",
            "how are you": "I'm doing great, thanks for asking! How about you?",
            "your name": "I'm CodeTrain Bot! I was built using the CodeTrain workflow framew
            "help": "I can chat about: CodeTrain, AI agents, or general topics. What interes
            "codetrain": "CodeTrain is a minimalist workflow framework inspired by PocketFlo
            "job": "A Job is a unit of work with three stages: receive_order, prepare_order,
            "hustle": "A Hustle orchestrates multiple Jobs into a workflow - like a project
            "manifest": "A Manifest is a shared dictionary that carries data between Jobs in
            "bye": "Goodbye! Have a great day!  ",
        }

        # Check for keyword matches
        for keyword, response in responses.items():
            if keyword in user_text:
                return response

        # Default responses
        defaults = [
            "That's interesting! Tell me more.",
            "I see. Can you elaborate?",
            "Fascinating! What else would you like to discuss?",
            "I'm learning from you!  ",
        ]
        import random
        return random.choice(defaults)
```

```
    def ship_order(self, manifest, user_input, response):
        """Display response and loop back for more input."""
        if user_input is None or response is None:
            print("\n" + "=" * 60)
            print("  Goodbye! Thanks for chatting!")
            print("=" * 60)
            return None  # End the conversation

        # Print the bot's response
        print(f"Bot: {response}")

        # Add bot response to history
        manifest["messages"].append({"role": "bot", "content": response})

        # Loop back to continue the conversation
        return "continue"


# Create the chat job
chat_job = SimpleChatJob()

# Set up self-looping: when job returns "continue", it loops back to itself
chat_job - "continue" >> chat_job

# Create the hustle
chat_hustle = ct.Hustle(start=chat_job)

# Start the chat
manifest = {}
chat_hustle.run(manifest)

# After the chat ends, you can access the full conversation history
print(f"\n  Conversation Stats:")
print(f"Total messages: {len(manifest.get('messages', []))}")
```

Line-by-Line Breakdown

Let's examine exactly what each part of this chatbot does:

Part 1: Imports and Class Definition

```
import codetrain as ct
```

What it does: Imports the CodeTrain library and gives it the short alias `ct` for easier typing.

```
class SimpleChatJob(ct.Job):
    """A simple chatbot with keyword responses."""
```

What it does: Defines a new class called `SimpleChatJob` that **inherits** from `ct.Job`. This means it gets all the built-in Job functionality (like the `run()` method) and we customize it with our own logic.

Part 2: The receive_order Method

```
def receive_order(self, manifest):
    """Initialize conversation history and get user input."""
```

What it does: Defines the first stage of our Job. This method receives the shared `manifest` dictionary and returns data that will be passed to `prepare_order`.

```
if "messages" not in manifest:
    manifest["messages"] = []
```

What it does: Checks if this is the first time the Job is running. If "messages" key doesn't exist in manifest, it creates an empty list. This only runs once at the start of the conversation.

```
    print("=" * 60)
    print("  Welcome to CodeTrain Chatbot!")
    print("=" * 60)
    print("Type 'exit' to end the conversation")
    print("-" * 60)
```

What it does: Displays a welcome banner when the chatbot starts. This only prints once because it's inside the `if` block that only runs on first iteration.

```
user_input = input("\nYou: ").strip()
```

What it does: Pauses execution and waits for the user to type something. The `input()` function displays the prompt `"\nYou: "` and returns whatever the user types. `.strip()` removes any extra spaces from the beginning and end.

```
if user_input.lower() == "exit":
    return None
```

What it does: Checks if the user typed "exit" (case-insensitive). If so, returns `None` which signals the Hustle to **stop the entire workflow** and end the chat.

```
manifest["messages"].append({"role": "user", "content": user_input})
```

What it does: Adds the user's message to the conversation history stored in the manifest. We store it as a dictionary with "role" and "content" keys (standard format for chat history).

```
return user_input
```

What it does: Returns the user's input text. This value gets passed as the argument to `prepare_order(user_input)`.

Part 3: The prepare_order Method

```
def prepare_order(self, user_input):
    """Generate a response based on keywords."""
```

What it does: Defines the second stage. This is where the "work" happens—generating a response based on the user's input.

```
if user_input is None:
    return None
```

What it does: Safety check. If user_input is None (which shouldn't happen, but just in case), return None to gracefully exit this stage.

```
user_text = user_input.lower()
```

What it does: Converts the user's input to lowercase so we can do case-insensitive matching. "Hello", "HELLO", and "hello" will all match the same keywords.

```
responses = {
    "hello": "Hello there! How can I help you today?  ",
    "hi": "Hi! What can I do for you?",
    # ... more responses
}
```

What it does: Creates a dictionary that maps **keywords** to **responses**. This is our chatbot's "brain"—it knows what to say when it sees certain words.

```
for keyword, response in responses.items():
    if keyword in user_text:
        return response
```

What it does: Loops through all keywords in our dictionary. If the keyword appears anywhere in the user's text, immediately return that response. As soon as we `return`, this method stops executing.

```
defaults = [
    "That's interesting! Tell me more.",
    "I see. Can you elaborate?",
    "Fascinating! What else would you like to discuss?",
    "I'm learning from you!  ",
```

```
]
import random
return random.choice(defaults)
```

What it does: If no keywords matched, pick a random default response. `random.choice()` selects one item randomly from the list, making the bot feel more natural.

Part 4: The ship_order Method

```
def ship_order(self, manifest, user_input, response):
    """Display response and loop back for more input."""
```

What it does: Defines the third and final stage. This method receives three things: the manifest, the original user_input (what came into prepare_order), and the response (what prepare_order returned).

```
if user_input is None or response is None:
    print("\n" + "=" * 60)
    print("  Goodbye! Thanks for chatting!")
    print("=" * 60)
    return None
```

What it does: Checks if we're exiting. If user_input is None (from the exit command), print a goodbye message and return None. Returning None tells the Hustle to **stop** the workflow.

```
print(f"Bot: {response}")
```

What it does: Displays the bot's response to the console. The `f` before the string allows us to insert the `response` variable directly into the text.

```
manifest["messages"].append({"role": "bot", "content": response})
```

What it does: Adds the bot's response to the conversation history in the manifest. Now the manifest contains both the user's message AND the bot's reply.

```
return "continue"
```

What it does: Returns the string `"continue"`. This is crucial—it's the **action name** that tells the Hustle where to go next.

Part 5: Setting Up the Workflow

```
chat_job = SimpleChatJob()
```

What it does: Creates an **instance** of our SimpleChatJob class. This is the actual object that will execute our three methods.

```
chat_job - "continue" >> chat_job
```

What it does: This is CodeTrain's magic syntax! It says: "When `chat_job` returns the action `'continue'`, send the workflow **back to the same job** (loop back)."

The `-` operator sets up a conditional connection, and `>>` points to the next job. Since both sides are `chat_job`, it creates a loop.

```
chat_hustle = ct.Hustle(start=chat_job)
```

What it does: Creates a Hustle object that will orchestrate our workflow. We tell it to start with `chat_job`. The Hustle manages the flow: calling receive_order → prepare_order → ship_order, then deciding where to go next based on the return value.

Part 6: Running the Chat

```
manifest = {}
```

What it does: Creates an empty dictionary that will be our manifest. This shared dictionary gets passed to every method and holds all our data (conversation history, settings, etc.).

```
chat_hustle.run(manifest)
```

What it does: Starts the workflow! The Hustle begins executing:

1. Calls `chat_job.receive_order(manifest)`
2. Takes the return value and passes it to `chat_job.prepare_order(return_value)`
3. Takes that result and calls `chat_job.ship_order(manifest, original_input, result)`
4. ship_order returns `"continue"`, so the Hustle loops back to step 1
5. This repeats until ship_order returns `None` (when user types "exit")

```
print(f"\n Conversation Stats:")
print(f"Total messages: {len(manifest.get('messages', []))}")
```

What it does: After the chat ends, we can still access the manifest! It contains the full conversation history. `manifest.get('messages', [])` safely gets the messages list (returns empty list if key doesn't exist), and `len()` counts how many messages were exchanged.

> **How Self-Looping Works**
>
> 1. **First Run**: `receive_order` initializes the manifest and gets the first user input
> 2. **Processing**: `prepare_order` generates a response
> 3. **Completion**: `ship_order` displays the response and returns `"continue"`
> 4. **Loop**: The hustle sees `"continue"` and routes back to the same job

5. **Persistence**: The manifest retains all messages between iterations
6. **Exit**: When user types "exit", `receive_order` returns `None`, ending the loop

Connecting to OpenRouter with API Keys

To make your chatbot smarter using AI (like GPT-4 or Claude), you need to connect to an LLM provider. We'll use **OpenRouter**, which gives you access to many AI models through a single API.

Step 1: Get Your OpenRouter API Key

1. Go to openrouter.ai
2. Sign up for a free account
3. Go to Settings → API Keys
4. Create a new key and copy it (starts with `sk-or-v1-...`)

Step 2: Create a .env File

Create a file named `.env` in your project folder:

```
# .env file - NEVER commit this to Git!
OPENROUTER_API_KEY=sk-or-v1-your-actual-key-here
OPENROUTER_MODEL=anthropic/claude-3.5-sonnet
```

⚠ Keep Your API Key Secret!

- Never share your API key
- Never commit `.env` to GitHub
- Add `.env` to your `.gitignore` file
- Treat it like a password

Step 3: Create config.py

Create a `config.py` file that reads the API key from `.env`:

```
# config.py
import os
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()

# Get API key from environment
OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
```

```
OPENROUTER_MODEL = os.getenv("OPENROUTER_MODEL", "anthropic/claude-3.5-sonnet")

# Validate that API key exists
if not OPENROUTER_API_KEY:
    raise ValueError(
        "OPENROUTER_API_KEY not found! "
        "Please create a .env file with your API key."
    )
```

Step 4: Create an LLM Utility

Create a utility file to call the LLM:

```
# utils/llm_client.py
import requests
from config import OPENROUTER_API_KEY, OPENROUTER_MODEL

def call_llm_simple(prompt):
    """Send a prompt to the LLM and return the response."""

    response = requests.post(
        url="https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {OPENROUTER_API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": OPENROUTER_MODEL,
            "messages": [{"role": "user", "content": prompt}],
            "temperature": 0.7,
            "max_tokens": 500
        }
    )

    # Check if request was successful
    response.raise_for_status()

    # Extract and return the response text
    return response.json()["choices"][0]["message"]["content"]


# For conversation history (chatbots)
def call_llm(messages):
    """Send conversation history to LLM and return response."""

    response = requests.post(
```

```
        url="https://openrouter.ai/api/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {OPENROUTER_API_KEY}",
            "Content-Type": "application/json",
        },
        json={
            "model": OPENROUTER_MODEL,
            "messages": messages,
            "temperature": 0.8,
            "max_tokens": 500
        }
    )

    response.raise_for_status()
    return response.json()["choices"][0]["message"]["content"]
```

Step 5: Update Your Chatbot to Use LLM

Now modify your chatbot to use AI responses:

```
import codetrain as ct
from utils.llm_client import call_llm

class AIChatJob(ct.Job):
    """Chatbot powered by OpenRouter LLM."""

    def receive_order(self, manifest):
        """Get user input and manage conversation history."""
        if "messages" not in manifest:
            manifest["messages"] = [
                {"role": "system", "content": "You are a helpful assistant."}
            ]
            print("  AI Chatbot Ready!")
            print("Type 'exit' to quit\n")

        user_input = input("You: ").strip()

        if user_input.lower() == "exit":
            return None

        # Add user message to history
        manifest["messages"].append(
            {"role": "user", "content": user_input}
        )

        return manifest["messages"]
```

```
    def prepare_order(self, messages):
        """Get AI response from LLM."""
        if messages is None:
            return None

        try:
            # Call OpenRouter API
            response = call_llm(messages)
            return response
        except Exception as e:
            return f"[Error: {str(e)}]"

    def ship_order(self, manifest, messages, response):
        """Display response and continue."""
        if messages is None or response is None:
            print("\n  Goodbye!")
            return None

        print(f"AI: {response}\n")

        # Add AI response to history
        manifest["messages"].append(
            {"role": "assistant", "content": response}
        )

        return "continue"


# Set up and run
chat_job = AIChatJob()
chat_job - "continue" >> chat_job

hustle = ct.Hustle(start=chat_job)
hustle.run({})
```

Project Structure

Your project should look like this:

```
my_chatbot/
   .env                  # API keys (NEVER commit!)
   .gitignore           # Should include .env
   config.py            # Configuration loader
   utils/
       __init__.py
       llm_client.py    # LLM API calls
```

```
chatbot.py           # Your chatbot
```

Installation Requirements

Add these to your project:

```
# Using uv (recommended)
uv pip install python-dotenv requests

# Or using pip
pip install python-dotenv requests
```

> **Testing Your Setup**
>
> Before running the chatbot, test your config:
>
> ```
> # test_config.py
> from utils.llm_client import call_llm_simple
>
> # Test the connection
> response = call_llm_simple("Say hello!")
> print(response)
> ```
>
> If this works, your API key is configured correctly!

7. Using CodeTrain with OpenCode

The CodeTrain framework includes a comprehensive **SKILL.md** file that teaches OpenCode how to build CodeTrain workflows for any purpose—from chatbots to data pipelines to weather apps.

> **The Agentic Coding Cheat Code**
>
> Remember the "Finished Cars vs Engine Blocks" analogy? The CodeTrain SKILL.md is your **cheat code** for building custom "Engine Blocks." Because CodeTrain's architecture is minimalist and well-defined, you can feed the SKILL.md to a "Finished Car" like OpenCode and have it generate perfect CodeTrain Jobs for any workflow you need.

What is the CodeTrain SKILL.md?

The SKILL.md is a special file that contains:

- **Agentic Coding Workflow**: 8-step process for building LLM systems
- **Core Abstractions**: Job, Hustle, Manifest patterns
- **Design Patterns**: Workflow, Agent, Map Reduce, RAG
- **OpenRouter Integration**: How to use LLMs with CodeTrain

- **Best Practices**: Common patterns and mistakes to avoid
- **Complete Examples**: Working code for various use cases

When you install this SKILL.md, OpenCode can help you build any CodeTrain application with just a prompt like:

```
/skill codetrain Build a workflow that fetches weather data and sends email alerts
```

Installing the CodeTrain Skill

There are two ways to use the CodeTrain SKILL.md with OpenCode:

Option 1: Global Installation (Recommended)

Install once and use across all your projects:

```
# Create the skills directory
mkdir -p ~/.config/opencode/skills/codetrain

# Download the SKILL.md from GitHub
curl -o ~/.config/opencode/skills/codetrain/SKILL.md \
  https://raw.githubusercontent.com/Speedykom/CodeTrain/main/SKILL.md
```

Now OpenCode will automatically use CodeTrain knowledge in any project!

Option 2: Project-Specific Installation

For a single project, copy SKILL.md to your project root:

```
# In your project directory
curl -o SKILL.md https://raw.githubusercontent.com/Speedykom/CodeTrain/main/SKILL.md
```

What the SKILL.md Covers

The CodeTrain SKILL.md teaches OpenCode:

1. Agentic Coding Steps

- Requirements gathering
- Hustle design with mermaid diagrams
- Manifest design (data contracts)
- Job design patterns
- Implementation best practices
- Optimization strategies
- Reliability patterns

2. Core Abstractions

```
Job (Base)
  receive_order(manifest) → Load data
  prepare_order(data) → Execute work
  ship_order(manifest, data, result) → Finalize

BatchJob (Job)
  Processes multiple items

Hustle
  Orchestrates connected jobs
```

3. **Design Patterns**

- **Workflow**: Chain jobs in sequence (`job_a >> job_b`)
- **Agent**: Dynamic decision-making with loops
- **Map Reduce**: Parallel processing then combine
- **RAG**: Retrieval-Augmented Generation

4. **OpenRouter Integration** All LLM calls use CodeTrain's built-in config:

```python
from codetrain import call_llm_simple

# Simple prompt
response = call_llm_simple("Your question here")

# Conversation history
from codetrain import call_llm
messages = [{"role": "user", "content": "Hello"}]
response = call_llm(messages)
```

Example: Building with OpenCode + CodeTrain

Once the skill is installed, you can build complex workflows conversationally:

Example 1: Weather Alert System

```
You: /skill codetrain Build a workflow that:

      1. Fetches weather data from an API
      2. Checks if temperature > 30°C
      3. Sends email alert if hot

OpenCode: [Uses SKILL.md to generate CodeTrain code with:
           - FetchWeatherJob for API calls
           - CheckTemperatureJob for conditional logic
           - SendEmailJob for notifications
           - Proper manifest design
           - Error handling and retries]
```

Example 2: Document Processor

```
You: /skill codetrain Create a document processing pipeline:
      - Load PDF files from a directory
      - Extract text using LLM
      - Summarize each document
      - Save summaries to database

OpenCode: [Generates complete workflow with:
           - BatchJob for processing multiple files
           - call_llm_simple() for text extraction
           - Database integration via utilities
           - Proper error handling]
```

Project Structure SKILL.md Recommends

When OpenCode generates CodeTrain code, it follows this structure:

```
my_project/
  main.py              # Entry point
  jobs.py              # Job definitions
  hustle.py            # Workflow orchestration
  utils/
      __init__.py
      call_llm.py      # LLM utilities
  .env                 # API keys (never commit!)
  pyproject.toml       # Dependencies
  docs/
      design.md        # Design documentation
```

Best Practices from SKILL.md

DO:

- Start with simple workflows, add complexity incrementally
- Design manifest schema before coding
- Use `call_llm_simple()` for most tasks
- Keep Jobs focused on single responsibilities
- Add logging for debugging
- Test utilities individually

DON'T:

- Hardcode API keys (use `.env` + `codetrain.config`)
- Put business logic in utilities
- Skip error handling
- Over-engineer solutions
- Repeat data in manifest (use references)

Exercise: Use CodeTrain SKILL.md

Task: Use the installed CodeTrain skill to build a workflow.

1. **Install the skill** (if not already done):

   ```
   mkdir -p ~/.config/opencode/skills/codetrain
   curl -o ~/.config/opencode/skills/codetrain/SKILL.md \
     https://raw.githubusercontent.com/Speedykom/CodeTrain/main/SKILL.md
   ```

2. **Start OpenCode** in your project directory:

   ```
   cd your_project
   opencode
   ```

3. **Build with a prompt**:

   ```
   /skill codetrain Build a chatbot that:

   - Greets users by name
   - Answers questions about CodeTrain
   - Remembers conversation history
   - Exits when user types 'bye'
   ```

4. **Review the generated code**:

5. - Check the manifest design
   - Verify Job structure (receive → prepare → ship)
   - Look for proper error handling
   - Test the workflow

6. **Iterate**: Ask OpenCode to add features:

   ```
   Add error handling for API failures
   Add a feature to save conversation to a file
   Make the chatbot use LLM for responses
   ```

Exercise: Build a FAQ Bot

Task

Create a chatbot that answers frequently asked questions about Python programming. The bot should:

1. Recognize common Python questions (list, dict, function, loop, etc.)
2. Provide helpful answers with code examples
3. Handle unknown questions gracefully
4. Allow users to ask multiple questions in one session

Starter Code

```
import codetrain as ct

# FAQ knowledge base
FAQ_KNOWLEDGE = {
    "list": """Lists store ordered collections:

    my_list = [1, 2, 3]
    my_list.append(4)  # Add item
    print(my_list[0])  # Access: 1
    """,

    "dict": """Dictionaries store key-value pairs:

    my_dict = {"name": "Kwame", "age": 25}
    print(my_dict["name"])  # Access: Kwame
    my_dict["city"] = "Accra"  # Add key
    """,

    # Add more FAQs here
}

# TODO: Create your FAQ chatbot using CodeTrain
# Hint: Use self-looping pattern with keyword matching

# class FAQChatJob(ct.Job): ...
# chat_job = FAQChatJob()
# chat_job - "continue" >> chat_job
# hustle = ct.Hustle(start=chat_job)
# hustle.run({})
```

Solution

```
import codetrain as ct

FAQ_KNOWLEDGE = {
    "list": """ **Python Lists**

Lists store ordered collections of items:

```python
# Creating lists
fruits = ["apple", "banana", "cherry"]
numbers = [1, 2, 3, 4, 5]

# Common operations
fruits.append("orange")      # Add item
print(fruits[0])             # Access: "apple"
print(fruits[-1])            # Last item: "orange"
fruits.remove("banana")      # Remove item
len(fruits)                  # Get length: 3
```

Lists are mutable (can be changed after creation)." " ",

"dict": """ **Python Dictionaries**

Dictionaries store key-value pairs:

```python
# Creating dictionaries
person = {
    "name": "Kwame",
    "age": 25,
    "city": "Accra"
}

# Accessing and modifying
print(person["name"])        # Access: "Kwame"
person["email"] = "kwame@example.com"  # Add key
person.get("phone", "N/A")   # Safe access

# Looping
for key, value in person.items():
    print(f"{key}: {value}")
```

Dictionary keys must be hashable (strings, numbers, tuples)." " ",

"function": """ **Python Functions**
```

Functions are reusable blocks of code:

```
# Defining functions
def greet(name):
    '''This function greets a person.'''
    return f"Hello, {name}!"

# Calling functions
message = greet("Ama")
print(message)  # Hello, Ama!

# Functions with default parameters
def power(base, exponent=2):
    return base ** exponent

print(power(3))      # 9 (3²)
print(power(2, 3))   # 8 (2³)
```

Use functions to avoid repeating code!“ “ “,

"loop": """ **Python Loops**

For loops - iterate over sequences:

```
for i in range(5):
    print(i)  # 0, 1, 2, 3, 4

fruits = ["apple", "banana"]
for fruit in fruits:
    print(fruit)
```

While loops - iterate while condition is true:

```
count = 0
while count < 5:
    print(count)
    count += 1
```

Loop control:

- `break` - exit loop immediately
- `continue` - skip to next iteration”“ “,

  “class”: “ “ “ **Python Classes**

Classes are blueprints for creating objects:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        return f"Hi, I'm {self.name}"

# Creating objects
person1 = Person("Kwame", 25)
person2 = Person("Ama", 30)

print(person1.greet())  # Hi, I'm Kwame
print(person2.age)      # 30
```

Classes bundle data (attributes) and behavior (methods).” “ “, }
class FAQChatJob(ct.Job): “ “ “A FAQ chatbot that answers Python questions.” “ ”

```
def receive_order(self, manifest):
    """Get user question."""
    if "questions_asked" not in manifest:
        manifest["questions_asked"] = 0
        print("=" * 60)
        print("  Python FAQ Bot")
        print("=" * 60)
        print("Ask me about: list, dict, function, loop, class")
        print("Type 'exit' to quit, 'topics' to see all topics")
        print("-" * 60)

    user_input = input("\n  Your question: ").strip()

    if user_input.lower() == "exit":
        return None

    manifest["current_question"] = user_input
    return user_input

def prepare_order(self, user_input):
    """Find answer in FAQ knowledge base."""
    if user_input is None:
        return None

    user_lower = user_input.lower()

    # Check for topics command
    if user_lower in ["topics", "help", "?"]:
```

```
        return {
            "type": "topics",
            "content": "Available topics: " + ", ".join(FAQ_KNOWLEDGE.keys())
        }

    # Search for matching topic
    for topic, answer in FAQ_KNOWLEDGE.items():
        if topic in user_lower:
            return {"type": "answer", "content": answer}

    # No match found
    return {
        "type": "unknown",
        "content": """  I don't have an answer for that yet.
```

Try asking about: list, dict, function, loop, or class Or type ‘topics’ to see all available topics.” “ ”
}

```
def ship_order(self, manifest, user_input, response):
    """Display answer and continue."""
    if user_input is None:
        print("\n" + "=" * 60)
        print(f"  Total questions answered: {manifest.get('questions_asked', 0)}")
        print("  Goodbye!")
        print("=" * 60)
        return None

    print(f"\n{response['content']}")
    manifest["questions_asked"] += 1

    return "continue"
```

Set up and run the chatbot

chat_job = FAQChatJob() chat_job - “continue” » chat_job
hustle = ct.Hustle(start=chat_job) manifest = {} hustle.run(manifest)

```
:::

### Exercise: Build a Data Processing Workflow

1. **Load Data**: Read a list of user dictionaries
2. **Validate**: Check that each user has required fields (name, email, age)
3. **Transform**: Calculate a "user tier" (bronze < 18, silver 18-
65, gold > 65)
4. **Save**: Store processed users in the manifest
```

````
## Starter Code

::: {.cell}
``` {.python .cell-code}
import codetrain as ct

# Sample data
users_data = [
    {"name": "Kwame", "email": "kwame@example.com", "age": 25},
    {"name": "Ama", "email": "ama@example.com", "age": 70},
    {"name": "Invalid User"},  # Missing fields
    {"name": "Kofi", "email": "kofi@example.com", "age": 15},
]

# TODO: Create your jobs here
# class LoadDataJob(ct.Job): ...
# class ValidateUserJob(ct.Job): ...
# class TransformUserJob(ct.Job): ...
# class SaveResultsJob(ct.Job): ...

# TODO: Build and run the workflow
# workflow = ...
# hustle = ct.Hustle(start=workflow)
# manifest = {"users": users_data}
# hustle.run(manifest)
# print(manifest)
````
````

Solution

```
import codetrain as ct

# Sample data
users_data = [
    {"name": "Kwame", "email": "kwame@example.com", "age": 25},
    {"name": "Ama", "email": "ama@example.com", "age": 70},
    {"name": "Invalid User"},  # Missing fields
    {"name": "Kofi", "email": "kofi@example.com", "age": 15},
]

class LoadDataJob(ct.Job):
    """Loads user data from manifest."""

    def receive_order(self, manifest):
```

8. Key Takeaways

What You Learned

1. **AI Agents** are autonomous systems that perceive, reason, act, and learn—going beyond traditional scripts
2. **Orchestration** coordinates multiple agents/tasks into reliable, scalable workflows
3. **CodeTrain** provides minimalist abstractions (Job, Hustle, Manifest) for building agentic workflows
4. **Three-Stage Jobs**: `receive_order` → `prepare_order` → `ship_order` mirror real-world logistics
5. **Workflow Composition**: Use `>>` for sequential flows and `- "action" >>` for conditional branching

When to Use CodeTrain

CodeTrain is ideal for:

- **Data processing pipelines** (ETL workflows)
- **AI-powered automation** (LLM-based workflows)
- **Business process automation** (order processing, approvals)
- **Microservice orchestration** (chaining API calls)
- **Any multi-step process** that needs reliability and clarity

Next Steps

1. **Experiment** with the examples in this lesson
2. **Build** your own simple workflow (e.g., a content moderation pipeline)
3. **Explore** advanced features:
    - `BatchJob` for processing multiple items
    - `AsyncJob` and `AsyncHustle` for concurrent operations
    - LLM integration via `config.py` and OpenRouter
4. **Read** the SKILL.md in the CodeTrain repository for best practices
5. **Practice** converting manual processes into automated workflows

Resources

- **CodeTrain GitHub**: https://github.com/Speedykom/CodeTrain
- **PocketFlow** (inspiration): https://github.com/The-Pocket/PocketFlow
- **Examples**: Check the `examples/` folder in the CodeTrain repo
- **SKILL.md**: Comprehensive guide for AI agents building CodeTrain workflows

---

Remember: The best way to learn agentic programming is to build something. Start small, iterate, and soon you'll be orchestrating complex AI workflows with confidence!